

TP 13

Coloration d'un graphe – Sujet

Coloration d'un graphe

D'après CCINP-TSI 2024.

Introduction

La coloration d'un graphe consiste à attribuer une couleur à chaque sommet. Un k -coloriage donne une couleur (qui peut par exemple être un entier entre 0 et $k - 1$, pour simplifier) à chaque sommet, tel que deux sommets adjacents soient de couleurs différentes. On cherche en général à minimiser le nombre de couleurs utilisées.

On peut appliquer ce problème au coloriage d'une carte de pays, ou d'un dessin d'ensemble (figure).

Ainsi le coloriage de la pince (ainsi que le premier graphe) est un k -coloriage valide. Néanmoins, il ne minimise pas le nombre de couleurs utilisées. Le second graphe, quant à lui utilise un nombre minimal de couleurs.

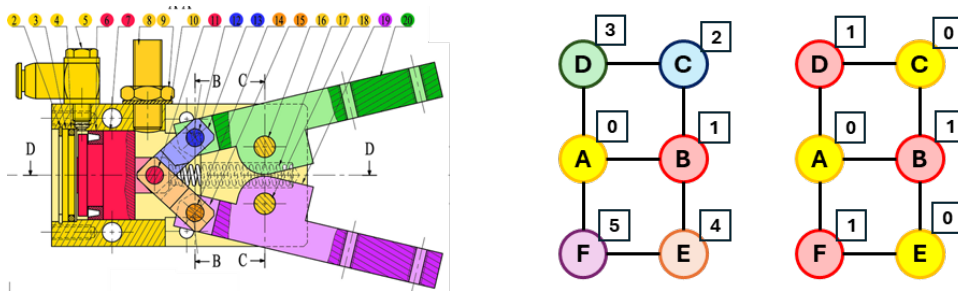


FIGURE 1 – Coloriage de la pince Schrader

On modélise le graphe par un dictionnaire d'adjacence. Les clés sont les sommets, modélisés par des chaînes de caractères correspondant aux classes d'équivalences cinématiques. Les valeurs sont des listes de chaînes de caractères.

Question 1 Donner le dictionnaire d'adjacence `Gex` associé au graphe de la figure 1.

Question 2 Implémenter la fonction `voisins(G:{}, s1:str, s2:str) -> bool` renvoyant `True` si `s1` et `s2` sont voisins, `False` sinon.

Question 3 Implémenter la fonction `liste_aretes(G:{}) -> [()]` qui renvoie la liste des arêtes du graphe. Une arête sera modélisée par un couple (tuple) de deux sommets. Le graphe étant non orienté, si `('A', 'B')` est dans la liste, `('B', 'A')` ne devra pas y figurer.

Question 4 Implémenter la fonction `is_graphe_correct(G:{}) -> bool` renvoyant `True` si, pour chaque arête `(s1,s2)` du graphe, si `s2` et `s1` sont voisins alors `s2` et `s1` sont voisins. La fonction renvoie `False` sinon.

Question 5 Vérifier que `Gex` est correctement implémenté. Le cas échéant, vérifier si le problème vient du graphe ou de la fonction `is_graphe_correct`.

Une k -coloration est représentée par un dictionnaire `C` pour lequel les clés sont les sommets des graphes et les valeurs sont des entiers (entre 0 et $k - 1$) qui représentent la couleur du sommet. Dans le cadre du graphe précédent, on a donc `C = {'A':0, 'B':1, 'C':0, 'D':1, 'E':0, 'F':1}`.

On souhaite vérifier si une coloration est valide, c'est-à-dire que deux sommets du graphe reliés par une arête ne peuvent pas avoir la même couleur.

Question 6 Implémenter la fonction `coloration_valide(G:{}, C:{}) -> bool` renvoyant `True` si la coloration est valide, `False` sinon. Tester votre fonction sur le dictionnaire de coloration précédent.

Algorithme intuitif de coloration

L'attribution des couleurs à chaque sommet est caractérisée par un dictionnaire `C` où `C[i]` est la couleur attribuée au sommet `i`. `C[i]` vaut `-1` si la couleur n'est pas encore attribuée. Le dictionnaire `C` ne contient initialement que des `-1` et ses valeurs sont modifiées progressivement, au fur et à mesure que les couleurs sont attribuées.

Question 7 Ecrire une fonction `init_c(G:{}) -> {}` permettant de créer un dictionnaire dont les clés sont identiques à celle du graphe. Ces clés sont associées à la valeur `-1`.

Question 8 Implémenter la fonction `colore_sommet(G:{}, C:{}, s:str) -> None`. Cette fonction ne renvoie rien mais modifie le dictionnaire `C` en donnant à `C[s]` la plus petite couleur possible, en fonction des couleurs des sommets voisins qui sont déjà colorés.

Par exemple, pour le graphe `Gex`, prenons `C = [{"A":0, "B":1, "C":-1, "D":-1, "E":-1, "F":-1}]`. Les sommets `"A"` et `"B"` ont donc déjà été colorés avec les couleurs `C["A"] = 0` et `C["B"] = 1`. L'appel `colore_sommet(G,C,"C")` modifie le dictionnaire `C` en `C = [{"A":0, "B":1, "C":0, "D":-1, "E":-1, "F":-1}]`. Cela veut dire que le sommet `"C"`, adjacent au sommet `"B"` mais pas au sommet `"A"`, a reçu la même couleur que le sommet `"A"`.

Question 9 Ecrire une fonction `colorer1` avec pour argument un dictionnaire `G` caractérisant un graphe, qui crée et renvoie le dictionnaire `C` des numéros des couleurs attribuées en colorant les sommets un par un.

Question 10 L'ordre de coloration imposé à la question précédente est arbitraire. On souhaite maintenant colorer le graphe en traitant les sommets selon un ordre défini, donné en argument. Écrire une fonction `colorer2` analogue à `colorer1` et avec un argument supplémentaire, une liste `ordre` fixant l'ordre de coloration des sommets.

Question 11 Donner le dictionnaire des couleurs renvoyé par `colorer2` pour colorer le graphe `Gex` donné en exemple en prenant `ordre = ['E', 'F', 'B', 'D', 'A', 'C']`.

Combien de couleurs ont été utilisées ?

La méthode que nous venons de décrire est rapide et fonctionne plutôt bien. Cependant, si on cherche à utiliser le nombre minimum de couleurs, l'efficacité de l'algorithme proposé ci-dessus dépend en grande partie de l'ordre dans lequel on choisit de colorer les sommets du graphe. L'objectif des sous parties suivantes est d'affiner la stratégie pour mieux choisir cet ordre de coloration.

Variante de Welsh-Powell

Une alternative est donnée par la variante de Welsh-Powell. L'idée est de parcourir l'ensemble des sommets du graphe par ordre décroissant de leurs degrés. Comme le degré d'un sommet est un entier positif, il est possible d'écrire un algorithme de tri efficace (dit par répartition).

Question 12 Écrire une fonction `degre` avec pour argument le dictionnaire d'adjacence `G` d'un graphe quelconque, qui renvoie la liste des degrés des sommets du graphe.

Question 13 Écrire une fonction `init` avec pour argument un entier `n`, qui renvoie une liste de listes `R` de taille `n`, telle que `R[i]` soit une liste vide. Par exemple, `init(3)` renverra `[[], [], []]`.

Question 14 Écrire une fonction `ranger` avec pour argument un dictionnaire d'adjacence `G`, qui renvoie une liste `R` de même taille que `G`, telle que `R[i]` soit la liste des sommets de degré `i`.

Question 15 Écrire une fonction `renverse` avec pour argument une liste `L`, qui crée et renvoie une nouvelle liste obtenue en lisant `L` dans l'ordre inverse. Par exemple, `renverse([1, 2, 3, 4])` renverra `[4, 3, 2, 1]`.

Question 16 Écrire une fonction `trier_sommets` avec pour argument un dictionnaire d'adjacence `G`, qui renvoie la liste des sommets triés dans l'ordre décroissant de leur degré.

Question 17 Pour un graphe à `n` sommets, quelle est la complexité temporelle de la fonction `trier_sommets` dans le pire des cas ?

Question 18 Écrire la fonction `colorer3` avec pour argument un dictionnaire d'adjacence `G`, qui crée et renvoie un dictionnaire de couleurs `C`, telle que `C[i]` soit la couleur à attribuer au sommet `i`, les sommets étant colorés dans l'ordre décroissant de leur degré.

Quelle est la complexité de `colorer3` dans le pire des cas pour un graphe à `n` sommets ?

Question 19 Pour le graphe `Gex`, donner la liste `C` des couleurs renvoyées par la fonction `colorer3`.

L'amélioration proposée donne de bons résultats dans un grand nombre de cas. Cependant, il reste quelques cas où la coloration obtenue utilise trop de couleurs.